C PROGRAMS: A BEGINNER'S LINE-BY-LINE STUDY GUIDE
Assignments p12.c through p19.c | prepared from the current workspace files
==

HOW TO USE THIS GUIDE
Each nonblank source line is shown with its original line number, followed
by an explanation. Blank source lines are skipped. Read the overview first,
then follow each program from input, through processing, to printed output.
The line notes explain syntax; the walkthroughs explain why the algorithm
works and how its steps lead to the requested result.

C IDEAS USED REPEATEDLY
- A variable is a named place to store a value. Its type (int, double, char)
  determines what kind of value fits there.
- An array stores several values of one type. C indexes start at 0.
- A string is a char array ending in '\0'. That final character marks its end.
- scanf reads formatted input; printf displays formatted output.
- &x means the address of x. scanf needs an address so it can fill x.
- A function packages steps under a name; return sends a value to its caller.
- A loop repeats work; an if chooses whether a block should run.
- { and } mark the beginning and end of a block.
- In matrix code, row and column indexes also begin at 0.

WHAT TO NOTICE WHEN READING AN ALGORITHM
1. Identify what input the program asks for and how it stores it.
2. Find the loop that visits each item, character, row, or column.
3. Find the condition that decides what action to take.
4. Trace one small example by hand and watch each variable change.
5. Find the final printf that displays the result.

==
p12.c — 94 source lines
==

  Purpose: Grades students' answers against an answer key. The central idea is t
o
  store one key character per question, compare every student's answer with the
  corresponding key character, and count matches. The score array is dynamically
  allocated because the number of students is entered while the program runs.

WALKTHROUGH / WHY THESE STEPS WORK
The program first asks for the number of questions and students.
It allocates an answer-key array and a score array of the needed
sizes. It reads the key once. Then, for each student, it compares
that student's answer at position i with the key at position i.
Equal characters add one point. Because calloc starts all scores
at zero, each score is ready to count matches. Finally, it prints
each student's total and frees both arrays.

Example: key A B C; student answers A D C. Compare positions:
A=A (+1), D!=B (+0), C=C (+1), so the score is 2 out of 3.
Input is case-sensitive: 'a' is different from 'A'.

LINE-BY-LINE NOTES
0001 | #include <stdio.h>
      Meaning: Imports stdio.h: standard input/output functions such as printf,
      scanf, fgets, and getchar.
0002 | #include <stdlib.h>

```
     Meaning: Imports stdlib.h: dynamic memory functions malloc, calloc, and
     free.
0004 | /*
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0005 |  * ALGORITHM - DRYCODE14
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0006 |  *
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0007 |  * START
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0008 |  *   Read m (number of questions) and n (number of students).
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0009 |  *   If m or n is not positive, report invalid input and stop.
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0010 |  *
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0011 |  *   Declare correctAnswers[0 ... m-1].
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0012 |  *   For item = 0 to m-1:
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0013 |  *       Read correctAnswers[item].
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0014 |  *
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0015 |  *   Declare scores[0 ... n-1].
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0016 |  *   For student = 0 to n-1:
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0017 |  *       scores[student] = 0
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0018 |  *       For item = 0 to m-1:
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0019 |  *           Read response
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0020 |  *           If response equals correctAnswers[item]:
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0021 |  *               scores[student] = scores[student] + 1
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0022 |  *
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0023 |  *   For student = 0 to n-1:
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
```

```
0024 |  *       Print student number and scores[student] out of m.
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0025 |  * STOP
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0026 |  *
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0027 |  * Input format: m n, then m answer characters for the key, followed by m
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0028 |  * response characters for each student. Answers can be any single
        character
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0029 |  * (for example, A, B, C, or D); comparison is case-sensitive.
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0030 |  */
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0032 | int main(void)
     Meaning: Defines main(): the program entry point; execution starts here.
0033 | {
     Meaning: Begins a block of statements.
0034 |         int m, n;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0036 |         // Get the number of test items and students before allocating
        arrays.
     Meaning: Comment: Get the number of test items and students before
     allocating arrays.
0037 |         printf("Enter number of items and students: ");
     Meaning: Prints a prompt or result: “Enter number of items and students: ”
     (format markers are filled by the arguments).
0038 |         if (scanf("%d %d", &m, &n) != 2 || m <= 0 || n <= 0) {
     Meaning: Checks whether scanf successfully read the requested input values
     .
0039 |                 printf("Invalid input: both counts must be positive
        integers.\n");
     Meaning: Prints a prompt or result: “Invalid input: both counts must be
     positive integers.\n” (format markers are filled by the arguments).
0040 |                 return 1;
     Meaning: Ends the program with an error status.
0041 |         }
     Meaning: Ends the current block.
0043 |         // The answer key stores one character for each test item.
     Meaning: Comment: The answer key stores one character for each test item.
0044 |         char *correctAnswers = malloc((size_t)m *
        sizeof(*correctAnswers));
     Meaning: Requests heap memory for the answer key; its size depends on the
     question count.
0046 |         // Each element stores the total correct answers for one student.
     Meaning: Comment: Each element stores the total correct answers for one
     student.
0047 |         int *scores = calloc((size_t)n, sizeof(*scores));
     Meaning: Requests zero-initialized heap memory for one score per student.
0048 |         if (correctAnswers == NULL || scores == NULL) {
     Meaning: Runs the following block only when this condition is true:
     (correctAnswers == NULL || scores == NULL) {
0049 |                 printf("Unable to allocate memory.\n");
```

```
      Meaning: Prints a prompt or result: “Unable to allocate memory.\n” (format
      markers are filled by the arguments).
0050 |                   free(correctAnswers);
      Meaning: Releases dynamically allocated memory so it is not left allocated
.
0051 |                   free(scores);
      Meaning: Releases dynamically allocated memory so it is not left allocated
.
0052 |                   return 1;
      Meaning: Ends the program with an error status.
0053 |               }
      Meaning: Ends the current block.
0055 |               // Read the answer key first, as required before evaluating
          students.
      Meaning: Comment: Read the answer key first, as required before evaluating
      students.
0056 |               printf("Enter the %d correct answers: ", m);
      Meaning: Prints a prompt or result: “Enter the %d correct answers: ” (form
at
      markers are filled by the arguments).
0057 |               for (int item = 0; item < m; item++) {
      Meaning: Repeats once for each question or answer position.
0058 |                   if (scanf(" %c", &correctAnswers[item]) != 1) {
      Meaning: Checks whether scanf successfully read the requested input values
.
0059 |                       printf("Could not read the answer key.\n");
      Meaning: Prints a prompt or result: “Could not read the answer key.\n”
      (format markers are filled by the arguments).
0060 |                       free(correctAnswers);
      Meaning: Releases dynamically allocated memory so it is not left allocated
.
0061 |                       free(scores);
      Meaning: Releases dynamically allocated memory so it is not left allocated
.
0062 |                       return 1;
      Meaning: Ends the program with an error status.
0063 |                   }
      Meaning: Ends the current block.
0064 |               }
      Meaning: Ends the current block.
0066 |               // Read and grade each student's complete set of responses in
          turn.
      Meaning: Comment: Read and grade each student's complete set of responses
in
      turn.
0067 |               for (int student = 0; student < n; student++) {
      Meaning: Repeats once for each student.
0068 |                   printf("Enter the %d responses for student %d: ", m,
          student + 1);
      Meaning: Prints a prompt or result: “Enter the %d responses for student %d
:
      ” (format markers are filled by the arguments).
0069 |                   for (int item = 0; item < m; item++) {
      Meaning: Repeats once for each question or answer position.
0070 |                       char response;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0071 |                       if (scanf(" %c", &response) != 1) {
      Meaning: Checks whether scanf successfully read the requested input values
.
0072 |                           printf("Could not read student
          responses.\n");
```

```
     Meaning: Prints a prompt or result: “Could not read student responses.\n”
     (format markers are filled by the arguments).
0073 |                                   free(correctAnswers);
     Meaning: Releases dynamically allocated memory so it is not left allocated
.
0074 |                                   free(scores);
     Meaning: Releases dynamically allocated memory so it is not left allocated
.
0075 |                                   return 1;
     Meaning: Ends the program with an error status.
0076 |                               }
     Meaning: Ends the current block.
0078 |                               // Award one point when this response matches the
        answer key.
     Meaning: Comment: Award one point when this response matches the answer ke
y.
0079 |                               if (response == correctAnswers[item]) {
     Meaning: Runs the following block only when this condition is true:
     (response == correctAnswers[item]) {
0080 |                                   scores[student]++;
     Meaning: Increases the loop index or count by one.
0081 |                               }
     Meaning: Ends the current block.
0082 |                           }
     Meaning: Ends the current block.
0083 |                   }
     Meaning: Ends the current block.
0085 |                   // Print the number correct for every student after grading is
        complete.
     Meaning: Comment: Print the number correct for every student after grading
     is complete.
0086 |                   printf("\nResults (score out of %d):\n", m);
     Meaning: Prints a prompt or result: “\nResults (score out of %d):\n” (form
at
     markers are filled by the arguments).
0087 |                   for (int student = 0; student < n; student++) {
     Meaning: Repeats once for each student.
0088 |                       printf("Student %d: %d\n", student + 1, scores[student]);
     Meaning: Prints a prompt or result: “Student %d: %d\n” (format markers are
     filled by the arguments).
0089 |                   }
     Meaning: Ends the current block.
0091 |                   free(correctAnswers);
     Meaning: Releases dynamically allocated memory so it is not left allocated
.
0092 |                   free(scores);
     Meaning: Releases dynamically allocated memory so it is not left allocated
.
0093 |                   return 0;
     Meaning: Ends the program successfully.
0094 | }
     Meaning: Ends the current block.

END OF FILE NOTES

================================================================================
==
p13.c — 370 source lines
================================================================================
==

  Purpose: A menu-driven matrix calculator. A Matrix structure keeps dimensions
```

  together with values. Reusable input and printing functions support addition,
  multiplication, transpose, and triangular checks. The input helper reads a who
le
  line and validates it before converting it.

WALKTHROUGH / WHY THESE STEPS WORK
Matrix dimensions are limited to 1..10, so the fixed arrays fit.
read_integer gets a whole line, checks the conversion and allowed
range, and retries invalid input. read_mat uses it for dimensions
and every entry. Each operation first checks its mathematical
dimension rule:
  Addition: same row and column counts.
  Multiplication: first columns = second rows.
  Transpose: swap row and column positions.
  Upper triangular: every cell below the diagonal is zero.
  Lower triangular: every cell above the diagonal is zero.
The menu's switch calls the matching operation. In triangular
checks the sum is printed only when the shape is triangular.

LINE-BY-LINE NOTES
0001 | #include <ctype.h>
      Meaning: Imports ctype.h: isspace for recognizing whitespace characters.
0002 | #include <errno.h>
      Meaning: Imports errno.h: errno and ERANGE for detecting an out-of-range
      integer conversion.
0003 | #include <stdbool.h>
      Meaning: Imports stdbool.h: the bool, true, and false names.
0004 | #include <stdio.h>
      Meaning: Imports stdio.h: standard input/output functions such as printf,
      scanf, fgets, and getchar.
0005 | #include <stdlib.h>
      Meaning: Imports stdlib.h: dynamic memory functions malloc, calloc, and
      free.
0006 | #include <string.h>
      Meaning: Imports string.h: strchr for checking whether an input line
      contains a newline.
0008 | #define MAX_SIZE 10
      Meaning: Sets the maximum matrix dimension to 10; arrays and input checks
      use this limit.
0009 | #define MAX_VALUE 1000000
      Meaning: Sets the accepted absolute matrix element limit to one million.
0011 | // Store a matrix's size and its values in one structure.
      Meaning: Comment: Store a matrix's size and its values in one structure.
0012 | typedef struct {
      Meaning: Starts a structure type so related matrix dimensions and data can
      be stored together.
0013 | int rows;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0014 | int cols;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0015 | long long values[MAX_SIZE][MAX_SIZE];
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0016 | } Matrix;
      Meaning: Names this structure type Matrix for later declarations.
0018 | // Read and validate one integer from a complete line of input.
      Meaning: Comment: Read and validate one integer from a complete line of
      input.
0019 | static bool read_integer(const char *prompt, long long min, long long
        max,

```
     Meaning: Defines read_integer(): reads one line, checks that it is an
     integer in range, and returns success/failure.
0020 |                           long long *value)
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0021 | {
     Meaning: Begins a block of statements.
0022 |     char line[128];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0024 |     for (;;) {
     Meaning: Repeats the block according to its initialization, condition, and
     update.
0025 |         char *end;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0026 |         long long parsed;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0028 |         // Ask for one complete line so invalid input can be retried
        safely.
     Meaning: Comment: Ask for one complete line so invalid input can be retrie
d
     safely.
0029 |         printf("%s", prompt);
     Meaning: Prints a prompt or result: “%s” (format markers are filled by the
     arguments).
0030 |         if (fgets(line, sizeof(line), stdin) == NULL) {
     Meaning: Checks for end-of-file or an input error while reading a line.
0031 |             return false;
     Meaning: Reports failure to the caller; often means the input could not be
     read.
0032 |         }
     Meaning: Ends the current block.
0034 |         // Remove extra characters if the entered line is too long.
     Meaning: Comment: Remove extra characters if the entered line is too long.
0035 |         if (strchr(line, '\n') == NULL && !feof(stdin)) {
     Meaning: Checks whether input has ended.
0036 |             int ch;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0037 |             while ((ch = getchar()) != '\n' && ch != EOF) {
     Meaning: Consumes characters until the end of the current line.
0038 |                 // Discard the rest of an overlong input line.
     Meaning: Comment: Discard the rest of an overlong input line.
0039 |             }
     Meaning: Ends the current block.
0040 |             printf("Input is too long. Please try again.\n");
     Meaning: Prints a prompt or result: “Input is too long. Please try again.\
n”
     (format markers are filled by the arguments).
0041 |             continue;
     Meaning: Skips the rest of this loop pass and starts the next pass.
0042 |         }
     Meaning: Ends the current block.
0044 |         // Convert the line and check that it contains only an allowed
        integer.
     Meaning: Comment: Convert the line and check that it contains only an
     allowed integer.
0045 |         errno = 0;
     Meaning: Stores the result of 0 in errno.
0046 |         parsed = strtoll(line, &end, 10);
```

     Meaning: Stores the result of strtoll(line, &end, 10) in parsed.
0047 | while (isspace((unsigned char)*end)) {
     Meaning: Skips whitespace so the next scan begins at a word.
0048 | end++;
     Meaning: Increases the loop index or count by one.
0049 | }
     Meaning: Ends the current block.
0051 | if (end == line || *end != '\0' || errno == ERANGE ||
     Meaning: Rejects invalid, overflowing, or out-of-range integer input.
0052 | parsed < min || parsed > max) {
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0053 | printf("Invalid input. Enter an integer from %lld to
        %lld.\n",
     Meaning: Prints a prompt or result: “Invalid input. Enter an integer from
     %lld to %lld.\n” (format markers are filled by the arguments).
0054 | min, max);
     Meaning: Executes this statement; the semicolon marks its end.
0055 | continue;
     Meaning: Skips the rest of this loop pass and starts the next pass.
0056 | }
     Meaning: Ends the current block.
0058 | // Return the validated value to the calling function.
     Meaning: Comment: Return the validated value to the calling function.
0059 | *value = parsed;
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0060 | return true;
     Meaning: Reports success to the caller.
0061 | }
     Meaning: Ends the current block.
0062 | }
     Meaning: Ends the current block.
0064 | // Read a dimension within the supported matrix size.
     Meaning: Comment: Read a dimension within the supported matrix size.
0065 | static bool read_dimension(const char *prompt, int *dimension)
     Meaning: Defines read_dimension(): gets a valid matrix dimension from 1
     through MAX_SIZE.
0066 | {
     Meaning: Begins a block of statements.
0067 | long long value;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0069 | if (!read_integer(prompt, 1, MAX_SIZE, &value)) {
     Meaning: Runs the following block only when this condition is true:
     (!read_integer(prompt, 1, MAX_SIZE, &value)) {
0070 | return false;
     Meaning: Reports failure to the caller; often means the input could not be
     read.
0071 | }
     Meaning: Ends the current block.
0073 | // Save the validated dimension as an int.
     Meaning: Comment: Save the validated dimension as an int.
0074 | *dimension = (int)value;
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0075 | return true;
     Meaning: Reports success to the caller.
0076 | }
     Meaning: Ends the current block.
0078 | // Read matrix dimensions and all of its elements.
     Meaning: Comment: Read matrix dimensions and all of its elements.

```
0079 | static bool read_mat(Matrix *matrix)
     Meaning: Defines read_mat(): reads matrix dimensions and every element int
o
     the Matrix structure.
0080 | {
     Meaning: Begins a block of statements.
0081 |     char prompt[80];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0083 |     if (!read_dimension("Enter the number of rows (1-10): ",
        &matrix->rows) ||
     Meaning: Runs the following block only when this condition is true:
     (!read_dimension("Enter the number of rows (1-10): ", &matrix->rows) ||
0084 |         !read_dimension("Enter the number of columns (1-10): ",
        &matrix->cols)) {
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0085 |         return false;
     Meaning: Reports failure to the caller; often means the input could not be
     read.
0086 |     }
     Meaning: Ends the current block.
0088 |     // Read every element using its row and column position.
     Meaning: Comment: Read every element using its row and column position.
0089 |     printf("Enter each matrix element separately when prompted.\n");
     Meaning: Prints a prompt or result: “Enter each matrix element separately
     when prompted.\n” (format markers are filled by the arguments).
0090 |     for (int row = 0; row < matrix->rows; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0091 |         for (int col = 0; col < matrix->cols; col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0092 |             long long value;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0094 |             snprintf(prompt, sizeof(prompt),
     Meaning: Builds a bounded prompt string with the current matrix row and
     column.
0095 |                      "Element [%d][%d] (-1000000 to 1000000): ",
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0096 |                      row + 1, col + 1);
     Meaning: Executes this statement; the semicolon marks its end.
0097 |             if (!read_integer(prompt, -MAX_VALUE, MAX_VALUE, &value)) {
     Meaning: Runs the following block only when this condition is true:
     (!read_integer(prompt, -MAX_VALUE, MAX_VALUE, &value)) {
0098 |                 return false;
     Meaning: Reports failure to the caller; often means the input could not be
     read.
0099 |             }
     Meaning: Ends the current block.
0100 |             matrix->values[row][col] = value;
     Meaning: Stores the result of value in matrix->values[row][col].
0101 |         }
     Meaning: Ends the current block.
0102 |     }
     Meaning: Ends the current block.
0104 |     return true;
     Meaning: Reports success to the caller.
0105 | }
     Meaning: Ends the current block.
0107 | // Display a matrix in rows and columns.
     Meaning: Comment: Display a matrix in rows and columns.
```

```
0108 | static void print_mat(const Matrix *matrix)
     Meaning: Defines print_mat(): prints the stored matrix row by row.
0109 | {
     Meaning: Begins a block of statements.
0110 |     for (int row = 0; row < matrix->rows; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0111 |         for (int col = 0; col < matrix->cols; col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0112 |             // Print each row on its own line.
     Meaning: Comment: Print each row on its own line.
0113 |             printf("%8lld", matrix->values[row][col]);
     Meaning: Prints a prompt or result: “%8lld” (format markers are filled by
     the arguments).
0114 |         }
     Meaning: Ends the current block.
0115 |         printf("\n");
     Meaning: Prints a prompt or result: “\n” (format markers are filled by the
     arguments).
0116 |     }
     Meaning: Ends the current block.
0117 | }
     Meaning: Ends the current block.
0119 | // Read two compatible matrices, add them, and display the result.
     Meaning: Comment: Read two compatible matrices, add them, and display the
     result.
0120 | static bool add_mat(void)
     Meaning: Defines add_mat(): reads two matrices, checks equal dimensions,
     adds corresponding entries, and prints the sum.
0121 | {
     Meaning: Begins a block of statements.
0122 |     Matrix first, second, result;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0124 |     printf("First matrix:\n");
     Meaning: Prints a prompt or result: “First matrix:\n” (format markers are
     filled by the arguments).
0125 |     if (!read_mat(&first)) {
     Meaning: Runs the following block only when this condition is true:
     (!read_mat(&first)) {
0126 |         return false;
     Meaning: Reports failure to the caller; often means the input could not be
     read.
0127 |     }
     Meaning: Ends the current block.
0128 |     printf("Second matrix:\n");
     Meaning: Prints a prompt or result: “Second matrix:\n” (format markers are
     filled by the arguments).
0129 |     if (!read_mat(&second)) {
     Meaning: Runs the following block only when this condition is true:
     (!read_mat(&second)) {
0130 |         return false;
     Meaning: Reports failure to the caller; often means the input could not be
     read.
0131 |     }
     Meaning: Ends the current block.
0133 |     if (first.rows != second.rows || first.cols != second.cols) {
     Meaning: Checks whether the two matrices have compatible dimensions.
0134 |         printf("Addition requires matrices with the same dimensions.\n");
     Meaning: Prints a prompt or result: “Addition requires matrices with the
     same dimensions.\n” (format markers are filled by the arguments).
0135 |         return true;
     Meaning: Reports success to the caller.
```

```
0136 |     }
     Meaning: Ends the current block.
0138 |     // Add elements that occupy the same row and column.
     Meaning: Comment: Add elements that occupy the same row and column.
0139 |     result.rows = first.rows;
     Meaning: Stores the result of first.rows in result.rows.
0140 |     result.cols = first.cols;
     Meaning: Stores the result of first.cols in result.cols.
0141 |     for (int row = 0; row < result.rows; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0142 |         for (int col = 0; col < result.cols; col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0143 |             result.values[row][col] =
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0144 |                 first.values[row][col] + second.values[row][col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0145 |         }
     Meaning: Ends the current block.
0146 |     }
     Meaning: Ends the current block.
0148 |     printf("Sum:\n");
     Meaning: Prints a prompt or result: “Sum:\n” (format markers are filled by
     the arguments).
0149 |     print_mat(&result);
     Meaning: Executes this statement; the semicolon marks its end.
0150 |     return true;
     Meaning: Reports success to the caller.
0151 | }
     Meaning: Ends the current block.
0153 | // Read two compatible matrices, multiply them, and display the result.
     Meaning: Comment: Read two compatible matrices, multiply them, and display
     the result.
0154 | static bool multiply_mat(void)
     Meaning: Defines multiply_mat(): reads two matrices, checks multiplication
     dimensions, computes each dot product, and prints it.
0155 | {
     Meaning: Begins a block of statements.
0156 |     Matrix first, second, result = {0};
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0158 |     printf("First matrix:\n");
     Meaning: Prints a prompt or result: “First matrix:\n” (format markers are
     filled by the arguments).
0159 |     if (!read_mat(&first)) {
     Meaning: Runs the following block only when this condition is true:
     (!read_mat(&first)) {
0160 |         return false;
     Meaning: Reports failure to the caller; often means the input could not be
     read.
0161 |     }
     Meaning: Ends the current block.
0162 |     printf("Second matrix:\n");
     Meaning: Prints a prompt or result: “Second matrix:\n” (format markers are
     filled by the arguments).
0163 |     if (!read_mat(&second)) {
     Meaning: Runs the following block only when this condition is true:
     (!read_mat(&second)) {
0164 |         return false;
     Meaning: Reports failure to the caller; often means the input could not be
     read.
```

```
0165 |     }
     Meaning: Ends the current block.
0167 |     if (first.cols != second.rows) {
     Meaning: Checks the multiplication rule: first columns must equal second
     rows.
0168 |         printf("Multiplication requires the first matrix's column count "
     Meaning: Prints a prompt or result: “Multiplication requires the first
     matrix's column count ” (format markers are filled by the arguments).
0169 |                "to equal the second matrix's row count.\n");
     Meaning: Executes this statement; the semicolon marks its end.
0170 |         return true;
     Meaning: Reports success to the caller.
0171 |     }
     Meaning: Ends the current block.
0173 |     // Each result element is the sum of one row-column pair of products.
     Meaning: Comment: Each result element is the sum of one row-column pair of
     products.
0174 |     result.rows = first.rows;
     Meaning: Stores the result of first.rows in result.rows.
0175 |     result.cols = second.cols;
     Meaning: Stores the result of second.cols in result.cols.
0176 |     for (int row = 0; row < result.rows; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0177 |         for (int col = 0; col < result.cols; col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0178 |             for (int k = 0; k < first.cols; k++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0179 |                 result.values[row][col] +=
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0180 |                     first.values[row][k] * second.values[k][col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0181 |             }
     Meaning: Ends the current block.
0182 |         }
     Meaning: Ends the current block.
0183 |     }
     Meaning: Ends the current block.
0185 |     printf("Product:\n");
     Meaning: Prints a prompt or result: “Product:\n” (format markers are fille
d
     by the arguments).
0186 |     print_mat(&result);
     Meaning: Executes this statement; the semicolon marks its end.
0187 |     return true;
     Meaning: Reports success to the caller.
0188 | }
     Meaning: Ends the current block.
0190 | // Read a matrix, transpose it, and display the transposed matrix.
     Meaning: Comment: Read a matrix, transpose it, and display the transposed
     matrix.
0191 | static bool trasp_mat(void)
     Meaning: Defines trasp_mat(): reads a matrix and writes each element at th
e
     transposed row/column.
0192 | {
     Meaning: Begins a block of statements.
0193 |     Matrix matrix, transpose;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0195 |     if (!read_mat(&matrix)) {
```

```
     Meaning: Runs the following block only when this condition is true:
     (!read_mat(&matrix)) {
0196 |          return false;
     Meaning: Reports failure to the caller; often means the input could not be
     read.
0197 |     }
     Meaning: Ends the current block.
0199 |     transpose.rows = matrix.cols;
     Meaning: Stores the result of matrix.cols in transpose.rows.
0200 |     transpose.cols = matrix.rows;
     Meaning: Stores the result of matrix.rows in transpose.cols.
0201 |     // Swap row and column positions to form the transpose.
     Meaning: Comment: Swap row and column positions to form the transpose.
0202 |     for (int row = 0; row < matrix.rows; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0203 |          for (int col = 0; col < matrix.cols; col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0204 |              transpose.values[col][row] = matrix.values[row][col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0205 |          }
     Meaning: Ends the current block.
0206 |     }
     Meaning: Ends the current block.
0208 |     printf("Transpose:\n");
     Meaning: Prints a prompt or result: “Transpose:\n” (format markers are
     filled by the arguments).
0209 |     print_mat(&transpose);
     Meaning: Executes this statement; the semicolon marks its end.
0210 |     return true;
     Meaning: Reports success to the caller.
0211 | }
     Meaning: Ends the current block.
0213 | // Read a square matrix and report whether it is upper triangular.
     Meaning: Comment: Read a square matrix and report whether it is upper
     triangular.
0214 | static bool uptr_mat(void)
     Meaning: Defines uptr_mat(): checks a square matrix for zeros below its
     diagonal and sums its entries.
0215 | {
     Meaning: Begins a block of statements.
0216 |     Matrix matrix;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0217 |     long long sum = 0;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0218 |     bool is_upper = true;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0220 |     if (!read_mat(&matrix)) {
     Meaning: Runs the following block only when this condition is true:
     (!read_mat(&matrix)) {
0221 |          return false;
     Meaning: Reports failure to the caller; often means the input could not be
     read.
0222 |     }
     Meaning: Ends the current block.
0223 |     if (matrix.rows != matrix.cols) {
     Meaning: Checks that the matrix is square before testing triangular form.
0224 |          printf("A triangular matrix must be square.\n");
     Meaning: Prints a prompt or result: “A triangular matrix must be square.\n
```

”
      (format markers are filled by the arguments).
0225 | return true;
      Meaning: Reports success to the caller.
0226 | }
      Meaning: Ends the current block.
0228 | // Sum all entries and check that entries below the diagonal are
         zero.
      Meaning: Comment: Sum all entries and check that entries below the diagona
l
      are zero.
0229 | for (int row = 0; row < matrix.rows; row++) {
      Meaning: Moves down matrix rows, stopping at the specified bound.
0230 | for (int col = 0; col < matrix.cols; col++) {
      Meaning: Moves across matrix columns, stopping at the specified bound.
0231 | sum += matrix.values[row][col];
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0232 | if (row > col && matrix.values[row][col] != 0) {
      Meaning: Checks for a nonzero entry below the main diagonal.
0233 | is_upper = false;
      Meaning: Stores the result of false in is_upper.
0234 | }
      Meaning: Ends the current block.
0235 | }
      Meaning: Ends the current block.
0236 | }
      Meaning: Ends the current block.
0238 | if (is_upper) {
      Meaning: Runs the following block only when this condition is true:
      (is_upper) {
0239 | printf("The matrix is upper triangular. Sum of all elements:
         %lld\n",
      Meaning: Prints a prompt or result: “The matrix is upper triangular. Sum o
f
      all elements: %lld\n” (format markers are filled by the arguments).
0240 | sum);
      Meaning: Executes this statement; the semicolon marks its end.
0241 | } else {
      Meaning: This line contributes to the surrounding C statement or expressio
n.
0242 | printf("The matrix is not upper triangular.\n");
      Meaning: Prints a prompt or result: “The matrix is not upper triangular.\n
”
      (format markers are filled by the arguments).
0243 | }
      Meaning: Ends the current block.
0244 | return true;
      Meaning: Reports success to the caller.
0245 | }
      Meaning: Ends the current block.
0247 | // Read a square matrix and report whether it is lower triangular.
      Meaning: Comment: Read a square matrix and report whether it is lower
      triangular.
0248 | static bool lowtr_mat(void)
      Meaning: Defines lowtr_mat(): checks a square matrix for zeros above its
      diagonal and sums its entries.
0249 | {
      Meaning: Begins a block of statements.
0250 | Matrix matrix;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.

```
0251 |     long long sum = 0;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0252 |     bool is_lower = true;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0254 |     if (!read_mat(&matrix)) {
     Meaning: Runs the following block only when this condition is true:
     (!read_mat(&matrix)) {
0255 |         return false;
     Meaning: Reports failure to the caller; often means the input could not be
     read.
0256 |     }
     Meaning: Ends the current block.
0257 |     if (matrix.rows != matrix.cols) {
     Meaning: Checks that the matrix is square before testing triangular form.
0258 |         printf("A triangular matrix must be square.\n");
     Meaning: Prints a prompt or result: “A triangular matrix must be square.\n
     ”
     (format markers are filled by the arguments).
0259 |         return true;
     Meaning: Reports success to the caller.
0260 |     }
     Meaning: Ends the current block.
0262 |     // Sum all entries and check that entries above the diagonal are
        zero.
     Meaning: Comment: Sum all entries and check that entries above the diagona
     l
     are zero.
0263 |     for (int row = 0; row < matrix.rows; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0264 |         for (int col = 0; col < matrix.cols; col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0265 |             sum += matrix.values[row][col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0266 |             if (row < col && matrix.values[row][col] != 0) {
     Meaning: Checks for a nonzero entry above the main diagonal.
0267 |                 is_lower = false;
     Meaning: Stores the result of false in is_lower.
0268 |             }
     Meaning: Ends the current block.
0269 |         }
     Meaning: Ends the current block.
0270 |     }
     Meaning: Ends the current block.
0272 |     if (is_lower) {
     Meaning: Runs the following block only when this condition is true:
     (is_lower) {
0273 |         printf("The matrix is lower triangular. Sum of all elements:
        %lld\n",
     Meaning: Prints a prompt or result: “The matrix is lower triangular. Sum o
     f
     all elements: %lld\n” (format markers are filled by the arguments).
0274 |                sum);
     Meaning: Executes this statement; the semicolon marks its end.
0275 |     } else {
     Meaning: This line contributes to the surrounding C statement or expressio
     n.
0276 |         printf("The matrix is not lower triangular.\n");
     Meaning: Prints a prompt or result: “The matrix is not lower triangular.\n
     ”
```

(format markers are filled by the arguments).
0277 | }
Meaning: Ends the current block.
0278 | return true;
Meaning: Reports success to the caller.
0279 | }
Meaning: Ends the current block.
0281 | // Present the menu and dispatch each operation to its function.
Meaning: Comment: Present the menu and dispatch each operation to its function.
0282 | int main(void)
Meaning: Defines main(): the program entry point; execution starts here.
0283 | {
Meaning: Begins a block of statements.
0284 | long long choice;
Meaning: Declares a variable or array; the type determines what values it can hold.
0286 | // Keep showing the menu until the user selects Exit or input ends.
Meaning: Comment: Keep showing the menu until the user selects Exit or input ends.
0287 | for (;;) {
Meaning: Repeats the block according to its initialization, condition, and update.
0288 | printf("\n--- Matrix Operations ---\n");
Meaning: Prints a prompt or result: “\n--- Matrix Operations ---\n” (format markers are filled by the arguments).
0289 | printf("1. Read and print two matrices\n");
Meaning: Prints a prompt or result: “1. Read and print two matrices\n” (format markers are filled by the arguments).
0290 | printf("2. Print a matrix\n");
Meaning: Prints a prompt or result: “2. Print a matrix\n” (format markers are filled by the arguments).
0291 | printf("3. Add two matrices\n");
Meaning: Prints a prompt or result: “3. Add two matrices\n” (format markers are filled by the arguments).
0292 | printf("4. Multiply two matrices\n");
Meaning: Prints a prompt or result: “4. Multiply two matrices\n” (format markers are filled by the arguments).
0293 | printf("5. Transpose a matrix\n");
Meaning: Prints a prompt or result: “5. Transpose a matrix\n” (format markers are filled by the arguments).
0294 | printf("6. Check upper triangular matrix and sum\n");
Meaning: Prints a prompt or result: “6. Check upper triangular matrix and sum\n” (format markers are filled by the arguments).
0295 | printf("7. Check lower triangular matrix and sum\n");
Meaning: Prints a prompt or result: “7. Check lower triangular matrix and sum\n” (format markers are filled by the arguments).
0296 | printf("0. Exit\n");
Meaning: Prints a prompt or result: “0. Exit\n” (format markers are filled by the arguments).
0298 | if (!read_integer("Choose an operation: ", 0, 7, &choice)) {
Meaning: Runs the following block only when this condition is true: (!read_integer("Choose an operation: ", 0, 7, &choice)) {
0299 | printf("\nInput ended. Exiting.\n");
Meaning: Prints a prompt or result: “\nInput ended. Exiting.\n” (format markers are filled by the arguments).
0300 | return 0;
Meaning: Ends the program successfully.
0301 | }

```
      Meaning: Ends the current block.
0303 |         // Call the function that matches the user's menu choice.
      Meaning: Comment: Call the function that matches the user's menu choice.
0304 |         switch (choice) {
      Meaning: Selects one menu-operation block based on the value of the
      expression.
0305 |             case 1: {
      Meaning: Begins the menu branch for this particular choice.
0306 |                 // Read and then display both matrices.
      Meaning: Comment: Read and then display both matrices.
0307 |                 Matrix first, second;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0308 |                 printf("First matrix:\n");
      Meaning: Prints a prompt or result: “First matrix:\n” (format markers are
      filled by the arguments).
0309 |                 if (!read_mat(&first)) {
      Meaning: Runs the following block only when this condition is true:
      (!read_mat(&first)) {
0310 |                     printf("\nInput ended. Exiting.\n");
      Meaning: Prints a prompt or result: “\nInput ended. Exiting.\n” (format
      markers are filled by the arguments).
0311 |                     return 0;
      Meaning: Ends the program successfully.
0312 |                 }
      Meaning: Ends the current block.
0313 |                 printf("Second matrix:\n");
      Meaning: Prints a prompt or result: “Second matrix:\n” (format markers are
      filled by the arguments).
0314 |                 if (!read_mat(&second)) {
      Meaning: Runs the following block only when this condition is true:
      (!read_mat(&second)) {
0315 |                     printf("\nInput ended. Exiting.\n");
      Meaning: Prints a prompt or result: “\nInput ended. Exiting.\n” (format
      markers are filled by the arguments).
0316 |                     return 0;
      Meaning: Ends the program successfully.
0317 |                 }
      Meaning: Ends the current block.
0318 |                 printf("First matrix:\n");
      Meaning: Prints a prompt or result: “First matrix:\n” (format markers are
      filled by the arguments).
0319 |                 print_mat(&first);
      Meaning: Executes this statement; the semicolon marks its end.
0320 |                 printf("Second matrix:\n");
      Meaning: Prints a prompt or result: “Second matrix:\n” (format markers are
      filled by the arguments).
0321 |                 print_mat(&second);
      Meaning: Executes this statement; the semicolon marks its end.
0322 |                 break;
      Meaning: Exits the current switch case or loop rather than falling through
      .
0323 |             }
      Meaning: Ends the current block.
0324 |             case 2: {
      Meaning: Begins the menu branch for this particular choice.
0325 |                 // Read and display one matrix.
      Meaning: Comment: Read and display one matrix.
0326 |                 Matrix matrix;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0327 |                 if (!read_mat(&matrix)) {
```

```
      Meaning: Runs the following block only when this condition is true:
      (!read_mat(&matrix)) {
0328 |                       printf("\nInput ended. Exiting.\n");
      Meaning: Prints a prompt or result: “\nInput ended. Exiting.\n” (format
      markers are filled by the arguments).
0329 |                       return 0;
      Meaning: Ends the program successfully.
0330 |                   }
      Meaning: Ends the current block.
0331 |                   printf("Matrix:\n");
      Meaning: Prints a prompt or result: “Matrix:\n” (format markers are filled
      by the arguments).
0332 |                   print_mat(&matrix);
      Meaning: Executes this statement; the semicolon marks its end.
0333 |                   break;
      Meaning: Exits the current switch case or loop rather than falling through
      .
0334 |               }
      Meaning: Ends the current block.
0335 |               case 3:
      Meaning: Begins the menu branch for this particular choice.
0336 |                   if (!add_mat()) {
      Meaning: Runs the following block only when this condition is true:
      (!add_mat()) {
0337 |                       printf("\nInput ended. Exiting.\n");
      Meaning: Prints a prompt or result: “\nInput ended. Exiting.\n” (format
      markers are filled by the arguments).
0338 |                       return 0;
      Meaning: Ends the program successfully.
0339 |                   }
      Meaning: Ends the current block.
0340 |                   break;
      Meaning: Exits the current switch case or loop rather than falling through
      .
0341 |               case 4:
      Meaning: Begins the menu branch for this particular choice.
0342 |                   if (!multiply_mat()) {
      Meaning: Runs the following block only when this condition is true:
      (!multiply_mat()) {
0343 |                       printf("\nInput ended. Exiting.\n");
      Meaning: Prints a prompt or result: “\nInput ended. Exiting.\n” (format
      markers are filled by the arguments).
0344 |                       return 0;
      Meaning: Ends the program successfully.
0345 |                   }
      Meaning: Ends the current block.
0346 |                   break;
      Meaning: Exits the current switch case or loop rather than falling through
      .
0347 |               case 5:
      Meaning: Begins the menu branch for this particular choice.
0348 |                   if (!trasp_mat()) {
      Meaning: Runs the following block only when this condition is true:
      (!trasp_mat()) {
0349 |                       printf("\nInput ended. Exiting.\n");
      Meaning: Prints a prompt or result: “\nInput ended. Exiting.\n” (format
      markers are filled by the arguments).
0350 |                       return 0;
      Meaning: Ends the program successfully.
0351 |                   }
      Meaning: Ends the current block.
0352 |                   break;
```

```
      Meaning: Exits the current switch case or loop rather than falling through
.
0353 |             case 6:
      Meaning: Begins the menu branch for this particular choice.
0354 |                 if (!uptr_mat()) {
      Meaning: Runs the following block only when this condition is true:
      (!uptr_mat()) {
0355 |                     printf("\nInput ended. Exiting.\n");
      Meaning: Prints a prompt or result: “\nInput ended. Exiting.\n” (format
      markers are filled by the arguments).
0356 |                     return 0;
      Meaning: Ends the program successfully.
0357 |                 }
      Meaning: Ends the current block.
0358 |                 break;
      Meaning: Exits the current switch case or loop rather than falling through
.
0359 |             case 7:
      Meaning: Begins the menu branch for this particular choice.
0360 |                 if (!lowtr_mat()) {
      Meaning: Runs the following block only when this condition is true:
      (!lowtr_mat()) {
0361 |                     printf("\nInput ended. Exiting.\n");
      Meaning: Prints a prompt or result: “\nInput ended. Exiting.\n” (format
      markers are filled by the arguments).
0362 |                     return 0;
      Meaning: Ends the program successfully.
0363 |                 }
      Meaning: Ends the current block.
0364 |                 break;
      Meaning: Exits the current switch case or loop rather than falling through
.
0365 |             case 0:
      Meaning: Begins the menu branch for this particular choice.
0366 |                 printf("Goodbye.\n");
      Meaning: Prints a prompt or result: “Goodbye.\n” (format markers are fille
d
      by the arguments).
0367 |                 return 0;
      Meaning: Ends the program successfully.
0368 |         }
      Meaning: Ends the current block.
0369 |     }
      Meaning: Ends the current block.
0370 | }
      Meaning: Ends the current block.

END OF FILE NOTES

================================================================================
==
p14.c — 151 source lines
================================================================================
==

  Purpose: The menu interface for the custom string library. It reads whole
  sentences (including spaces), dispatches the selected operation, and prints th
e
  result. The implementation of each string operation is in mystring.h, which is
  included as a local header.

WALKTHROUGH / WHY THESE STEPS WORK
```

fgets reads a whole sentence, unlike %s, which stops at spaces.
The menu itself is a number, so scanf reads that number and
clear_input_line removes the remaining newline before fgets runs.
The switch chooses the operation. For copy, compare, reverse,
search, counting, and tokenization, p14.c calls the matching
function in mystring.h. The join result has a larger array because
both input strings may need to fit in it.

LINE-BY-LINE NOTES
```
0001 | #include <stdio.h>
     Meaning: Imports stdio.h: standard input/output functions such as printf,
     scanf, fgets, and getchar.
0002 | #include "mystring.h"
     Meaning: Imports mystring.h: makes declarations from this header available
.
0004 | // Read a complete line, including spaces, within the supported size.
     Meaning: Comment: Read a complete line, including spaces, within the
     supported size.
0005 | int read_string(const char prompt[], char text[])
     Meaning: Defines read_string(): reads one whole line of text, preserving
     spaces and enforcing the buffer limit.
0006 | {
     Meaning: Begins a block of statements.
0007 |     int ch;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0009 |     while (1) {
     Meaning: Repeats the block while the stated condition remains true.
0010 |         int i = 0;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0011 |         printf("%s", prompt);
     Meaning: Prints a prompt or result: “%s” (format markers are filled by the
     arguments).
0012 |         if (fgets(text, MYSTRING_MAX, stdin) == NULL) {
     Meaning: Checks for end-of-file or an input error while reading a line.
0013 |             return 0;
     Meaning: Ends the program successfully.
0014 |         }
     Meaning: Ends the current block.
0016 |         while (text[i] != '\0' && text[i] != '\n') {
     Meaning: Repeats the block while the stated condition remains true.
0017 |             i++;
     Meaning: Increases the loop index or count by one.
0018 |         }
     Meaning: Ends the current block.
0019 |         if (text[i] == '\n') {
     Meaning: Runs the following block only when this condition is true: (text[
i]
     == '\n') {
0020 |             text[i] = '\0';
     Meaning: Stores the result of '\0' in text[i].
0021 |             return 1;
     Meaning: Ends the program with an error status.
0022 |         }
     Meaning: Ends the current block.
0024 |         ch = getchar();
     Meaning: Stores the result of getchar() in ch.
0025 |         if (ch == '\n' || ch == EOF) {
     Meaning: Checks whether the input line ended or reached end-of-file.
0026 |             return 1;
     Meaning: Ends the program with an error status.
```

```
0027 |         }
     Meaning: Ends the current block.
0029 |         while (ch != '\n' && ch != EOF) {
     Meaning: Repeats the block while the stated condition remains true.
0030 |             ch = getchar();
     Meaning: Stores the result of getchar() in ch.
0031 |         }
     Meaning: Ends the current block.
0032 |         printf("String is too long. Enter fewer than %d characters.\n",
     Meaning: Prints a prompt or result: “String is too long. Enter fewer than
%d
     characters.\n” (format markers are filled by the arguments).
0033 |                MYSTRING_MAX);
     Meaning: Executes this statement; the semicolon marks its end.
0034 |     }
     Meaning: Ends the current block.
0035 | }
     Meaning: Ends the current block.
0037 | // Remove the rest of the menu input line after reading its number.
     Meaning: Comment: Remove the rest of the menu input line after reading its
     number.
0038 | void clear_input_line(void)
     Meaning: Defines clear_input_line(): consumes leftover characters through
     the end of the current input line.
0039 | {
     Meaning: Begins a block of statements.
0040 |     int ch;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0041 |     while ((ch = getchar()) != '\n' && ch != EOF) {
     Meaning: Consumes characters until the end of the current line.
0042 |     }
     Meaning: Ends the current block.
0043 | }
     Meaning: Ends the current block.
0045 | // Display the menu and run the selected string operation.
     Meaning: Comment: Display the menu and run the selected string operation.
0046 | int main(void)
     Meaning: Defines main(): the program entry point; execution starts here.
0047 | {
     Meaning: Begins a block of statements.
0048 |     int choice;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0049 |     char first[MYSTRING_MAX];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0050 |     char second[MYSTRING_MAX];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0051 |     char result[MYSTRING_MAX * 2];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0053 |     while (1) {
     Meaning: Repeats the block while the stated condition remains true.
0054 |         printf("\n--- String Operations ---\n");
     Meaning: Prints a prompt or result: “\n--- String Operations ---\n” (forma
t
     markers are filled by the arguments).
0055 |         printf("1. Find string length\n");
     Meaning: Prints a prompt or result: “1. Find string length\n” (format
     markers are filled by the arguments).
```

```
0056 |         printf("2. Copy a string\n");
     Meaning: Prints a prompt or result: “2. Copy a string\n” (format markers a
re
     filled by the arguments).
0057 |         printf("3. Join two strings\n");
     Meaning: Prints a prompt or result: “3. Join two strings\n” (format marker
s
     are filled by the arguments).
0058 |         printf("4. Compare two strings\n");
     Meaning: Prints a prompt or result: “4. Compare two strings\n” (format
     markers are filled by the arguments).
0059 |         printf("5. Reverse a string\n");
     Meaning: Prints a prompt or result: “5. Reverse a string\n” (format marker
s
     are filled by the arguments).
0060 |         printf("6. Search for a substring\n");
     Meaning: Prints a prompt or result: “6. Search for a substring\n” (format
     markers are filled by the arguments).
0061 |         printf("7. Count vowels and consonants\n");
     Meaning: Prints a prompt or result: “7. Count vowels and consonants\n”
     (format markers are filled by the arguments).
0062 |         printf("8. Split a string into words\n");
     Meaning: Prints a prompt or result: “8. Split a string into words\n” (form
at
     markers are filled by the arguments).
0063 |         printf("0. Exit\n");
     Meaning: Prints a prompt or result: “0. Exit\n” (format markers are filled
     by the arguments).
0064 |         printf("Choose an operation: ");
     Meaning: Prints a prompt or result: “Choose an operation: ” (format marker
s
     are filled by the arguments).
0066 |         if (scanf("%d", &choice) != 1) {
     Meaning: Checks whether scanf successfully read the requested input values
.
0067 |             printf("Please enter a menu number.\n");
     Meaning: Prints a prompt or result: “Please enter a menu number.\n” (forma
t
     markers are filled by the arguments).
0068 |             if (feof(stdin)) {
     Meaning: Checks whether input has ended.
0069 |                 return 0;
     Meaning: Ends the program successfully.
0070 |             }
     Meaning: Ends the current block.
0071 |             clear_input_line();
     Meaning: Executes this statement; the semicolon marks its end.
0072 |             continue;
     Meaning: Skips the rest of this loop pass and starts the next pass.
0073 |         }
     Meaning: Ends the current block.
0074 |         clear_input_line();
     Meaning: Executes this statement; the semicolon marks its end.
0076 |         switch (choice) {
     Meaning: Selects one menu-operation block based on the value of the
     expression.
0077 |             case 1:
     Meaning: Begins the menu branch for this particular choice.
0078 |                 if (!read_string("Enter a string: ", first)) return 0;
     Meaning: Runs the following block only when this condition is true:
     (!read_string("Enter a string: ", first)) return 0;
0079 |                 printf("Length: %d\n", my_strlen(first));
```

```
      Meaning: Prints a prompt or result: “Length: %d\n” (format markers are
      filled by the arguments).
0080 |                   break;
      Meaning: Exits the current switch case or loop rather than falling through
.
0082 |               case 2:
      Meaning: Begins the menu branch for this particular choice.
0083 |                   if (!read_string("Enter a string to copy: ", first))
         return 0;
      Meaning: Runs the following block only when this condition is true:
      (!read_string("Enter a string to copy: ", first)) return 0;
0084 |                   my_strcopy(second, first);
      Meaning: Executes this statement; the semicolon marks its end.
0085 |                   printf("Copied string: %s\n", second);
      Meaning: Prints a prompt or result: “Copied string: %s\n” (format markers
      are filled by the arguments).
0086 |                   break;
      Meaning: Exits the current switch case or loop rather than falling through
.
0088 |               case 3:
      Meaning: Begins the menu branch for this particular choice.
0089 |                   if (!read_string("Enter the first string: ", first))
         return 0;
      Meaning: Runs the following block only when this condition is true:
      (!read_string("Enter the first string: ", first)) return 0;
0090 |                   if (!read_string("Enter the second string: ", second))
         return 0;
      Meaning: Runs the following block only when this condition is true:
      (!read_string("Enter the second string: ", second)) return 0;
0091 |                   my_strcopy(result, first);
      Meaning: Executes this statement; the semicolon marks its end.
0092 |                   my_strconcat(result, second);
      Meaning: Executes this statement; the semicolon marks its end.
0093 |                   printf("Joined string: %s\n", result);
      Meaning: Prints a prompt or result: “Joined string: %s\n” (format markers
      are filled by the arguments).
0094 |                   break;
      Meaning: Exits the current switch case or loop rather than falling through
.
0096 |               case 4: {
      Meaning: Begins the menu branch for this particular choice.
0097 |                   int comparison;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0098 |                   if (!read_string("Enter the first string: ", first))
         return 0;
      Meaning: Runs the following block only when this condition is true:
      (!read_string("Enter the first string: ", first)) return 0;
0099 |                   if (!read_string("Enter the second string: ", second))
         return 0;
      Meaning: Runs the following block only when this condition is true:
      (!read_string("Enter the second string: ", second)) return 0;
0100 |                   comparison = my_strcompare(first, second);
      Meaning: Stores the result of my_strcompare(first, second) in comparison.
0101 |                   if (comparison == 0) {
      Meaning: Chooses the comparison message based on the comparison result.
0102 |                       printf("The strings are equal.\n");
      Meaning: Prints a prompt or result: “The strings are equal.\n” (format
      markers are filled by the arguments).
0103 |                   } else if (comparison < 0) {
      Meaning: This line contributes to the surrounding C statement or expressio
n.
```

```
0104 |                     printf("The first string is lexicographically
        smaller.\n");
      Meaning: Prints a prompt or result: “The first string is lexicographically
      smaller.\n” (format markers are filled by the arguments).
0105 |                 } else {
      Meaning: This line contributes to the surrounding C statement or expressio
n.
0106 |                     printf("The second string is lexicographically
        smaller.\n");
      Meaning: Prints a prompt or result: “The second string is lexicographicall
y
      smaller.\n” (format markers are filled by the arguments).
0107 |                 }
      Meaning: Ends the current block.
0108 |                 break;
      Meaning: Exits the current switch case or loop rather than falling through
.
0109 |             }
      Meaning: Ends the current block.
0111 |             case 5:
      Meaning: Begins the menu branch for this particular choice.
0112 |                 if (!read_string("Enter a string: ", first)) return 0;
      Meaning: Runs the following block only when this condition is true:
      (!read_string("Enter a string: ", first)) return 0;
0113 |                 my_strreverse(first);
      Meaning: Executes this statement; the semicolon marks its end.
0114 |                 printf("Reversed string: %s\n", first);
      Meaning: Prints a prompt or result: “Reversed string: %s\n” (format marker
s
      are filled by the arguments).
0115 |                 break;
      Meaning: Exits the current switch case or loop rather than falling through
.
0117 |             case 6: {
      Meaning: Begins the menu branch for this particular choice.
0118 |                 int position;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0119 |                 if (!read_string("Enter the main string: ", first))
        return 0;
      Meaning: Runs the following block only when this condition is true:
      (!read_string("Enter the main string: ", first)) return 0;
0120 |                 if (!read_string("Enter the substring to find: ",
        second)) return 0;
      Meaning: Runs the following block only when this condition is true:
      (!read_string("Enter the substring to find: ", second)) return 0;
0121 |                 position = my_strsearch(first, second);
      Meaning: Stores the result of my_strsearch(first, second) in position.
0122 |                 if (position == 0) {
      Meaning: Checks whether the requested substring was absent.
0123 |                     printf("Substring not found.\n");
      Meaning: Prints a prompt or result: “Substring not found.\n” (format marke
rs
      are filled by the arguments).
0124 |                 } else {
      Meaning: This line contributes to the surrounding C statement or expressio
n.
0125 |                     printf("Substring starts at position %d.\n",
        position);
      Meaning: Prints a prompt or result: “Substring starts at position %d.\n”
      (format markers are filled by the arguments).
0126 |                 }
```

      Meaning: Ends the current block.
0127 | break;
      Meaning: Exits the current switch case or loop rather than falling through
.
0128 | }
      Meaning: Ends the current block.
0130 | case 7: {
      Meaning: Begins the menu branch for this particular choice.
0131 | int vowels, consonants;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0132 | if (!read_string("Enter a string: ", first)) return 0;
      Meaning: Runs the following block only when this condition is true:
      (!read_string("Enter a string: ", first)) return 0;
0133 | my_count_vowels_consonants(first, &vowels, &consonants);
      Meaning: Executes this statement; the semicolon marks its end.
0134 | printf("Vowels: %d\nConsonants: %d\n", vowels,
         consonants);
      Meaning: Prints a prompt or result: “Vowels: %d\nConsonants: %d\n” (format
      markers are filled by the arguments).
0135 | break;
      Meaning: Exits the current switch case or loop rather than falling through
.
0136 | }
      Meaning: Ends the current block.
0138 | case 8:
      Meaning: Begins the menu branch for this particular choice.
0139 | if (!read_string("Enter a string: ", first)) return 0;
      Meaning: Runs the following block only when this condition is true:
      (!read_string("Enter a string: ", first)) return 0;
0140 | my_strtokenize(first);
      Meaning: Executes this statement; the semicolon marks its end.
0141 | break;
      Meaning: Exits the current switch case or loop rather than falling through
.
0143 | case 0:
      Meaning: Begins the menu branch for this particular choice.
0144 | printf("Goodbye.\n");
      Meaning: Prints a prompt or result: “Goodbye.\n” (format markers are fille
d
      by the arguments).
0145 | return 0;
      Meaning: Ends the program successfully.
0147 | default:
      Meaning: Handles any menu value not matched by the listed cases.
0148 | printf("Choose a number from 0 to 8.\n");
      Meaning: Prints a prompt or result: “Choose a number from 0 to 8.\n” (form
at
      markers are filled by the arguments).
0149 | }
      Meaning: Ends the current block.
0150 | }
      Meaning: Ends the current block.
0151 | }
      Meaning: Ends the current block.

END OF FILE NOTES

==

  Purpose: A small custom string library. Each function walks a character array
  until the terminating '\0'; none of the requested length, copy, join,
  comparison, or reverse tasks uses the prohibited standard function.

WALKTHROUGH / HOW THE CHARACTER LOOPS WORK
A C string ends at '\0'. Each function advances an index until it
reaches that marker. Length counts the characters before it;
copy transfers them and also copies '\0'; concatenation first
finds the destination end, then copies the source there; compare
stops at the first different character; reverse swaps the first
and last characters and moves inward.
Search tries each possible starting position and checks characters
against the substring. A returned position is one-based for people
(first position = 1); 0 means absent. Counting includes only
English letters, not punctuation or spaces. Tokenization skips
spaces and prints each run of non-space characters.

LINE-BY-LINE NOTES
0001 | #ifndef MYSTRING_H
      Meaning: This line contributes to the surrounding C statement or expressio
n.
0002 | #define MYSTRING_H
      Meaning: Defines a named constant used elsewhere in this program.
0004 | #include <stdio.h>
      Meaning: Imports stdio.h: standard input/output functions such as printf,
      scanf, fgets, and getchar.
0006 | #define MYSTRING_MAX 200
      Meaning: Sets the shared maximum size for strings handled by the custom
      library.
0008 | // Return the number of characters before the string's terminating null.
      Meaning: Comment: Return the number of characters before the string's
      terminating null.
0009 | int my_strlen(const char text[])
      Meaning: Defines my_strlen(): counts characters until the string terminato
r.
0010 | {
      Meaning: Begins a block of statements.
0011 | int length = 0;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0012 | while (text[length] != '\0') {
      Meaning: Repeats the block while the stated condition remains true.
0013 | length++;
      Meaning: Increases the loop index or count by one.
0014 | }
      Meaning: Ends the current block.
0015 | return length;
      Meaning: Sends this result back to the function's caller.
0016 | }
      Meaning: Ends the current block.
0018 | // Copy a string into another character array.
      Meaning: Comment: Copy a string into another character array.
0019 | void my_strcopy(char destination[], const char source[])
      Meaning: Defines my_strcopy(): copies characters through the terminator in
to
      the destination.
0020 | {
      Meaning: Begins a block of statements.
0021 | int i = 0;
      Meaning: Declares a variable or array; the type determines what values it

```
     can hold.
0022 |     while (source[i] != '\0') {
     Meaning: Continues copying/appending until the string terminator is reache
d.
0023 |         destination[i] = source[i];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0024 |         i++;
     Meaning: Increases the loop index or count by one.
0025 |     }
     Meaning: Ends the current block.
0026 |     destination[i] = '\0';
     Meaning: Stores the result of '\0' in destination[i].
0027 | }
     Meaning: Ends the current block.
0029 | // Add the source string to the end of the destination string.
     Meaning: Comment: Add the source string to the end of the destination
     string.
0030 | void my_strconcat(char destination[], const char source[])
     Meaning: Defines my_strconcat(): finds the destination end, then appends
     source characters.
0031 | {
     Meaning: Begins a block of statements.
0032 |     int end = my_strlen(destination);
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0033 |     int i = 0;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0035 |     while (source[i] != '\0') {
     Meaning: Continues copying/appending until the string terminator is reache
d.
0036 |         destination[end + i] = source[i];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0037 |         i++;
     Meaning: Increases the loop index or count by one.
0038 |     }
     Meaning: Ends the current block.
0039 |     destination[end + i] = '\0';
     Meaning: Stores the result of '\0' in destination[end + i].
0040 | }
     Meaning: Ends the current block.
0042 | // Return 0 for equal strings, a negative value if first comes first,
        else positive.
     Meaning: Comment: Return 0 for equal strings, a negative value if first
     comes first, else positive.
0043 | int my_strcompare(const char first[], const char second[])
     Meaning: Defines my_strcompare(): compares corresponding character codes a
nd
     returns their difference.
0044 | {
     Meaning: Begins a block of statements.
0045 |     int i = 0;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0047 |     while (first[i] != '\0' && second[i] != '\0' &&
     Meaning: Repeats the block while the stated condition remains true.
0048 |            first[i] == second[i]) {
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0049 |         i++;
```

     Meaning: Increases the loop index or count by one.
0050 | }
     Meaning: Ends the current block.
0051 | return (unsigned char)first[i] - (unsigned char)second[i];
     Meaning: Sends this result back to the function's caller.
0052 | }
     Meaning: Ends the current block.
0054 | // Reverse a string by swapping characters from its two ends.
     Meaning: Comment: Reverse a string by swapping characters from its two end
s.
0055 | void my_strreverse(char text[])
     Meaning: Defines my_strreverse(): swaps characters from opposite ends unti
l
     they meet.
0056 | {
     Meaning: Begins a block of statements.
0057 | int left = 0;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0058 | int right = my_strlen(text) - 1;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0060 | while (left < right) {
     Meaning: Keeps swapping inward-facing characters until the string is
     reversed.
0061 | char temporary = text[left];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0062 | text[left] = text[right];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0063 | text[right] = temporary;
     Meaning: Stores the result of temporary in text[right].
0064 | left++;
     Meaning: Increases the loop index or count by one.
0065 | right--;
     Meaning: Executes this statement; the semicolon marks its end.
0066 | }
     Meaning: Ends the current block.
0067 | }
     Meaning: Ends the current block.
0069 | // Find a substring and return its one-based starting position, or 0 if
        absent.
     Meaning: Comment: Find a substring and return its one-based starting
     position, or 0 if absent.
0070 | int my_strsearch(const char text[], const char substring[])
     Meaning: Defines my_strsearch(): tries each possible start position and
     returns the first match position.
0071 | {
     Meaning: Begins a block of statements.
0072 | int text_length = my_strlen(text);
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0073 | int sub_length = my_strlen(substring);
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0075 | if (sub_length == 0) {
     Meaning: Runs the following block only when this condition is true:
     (sub_length == 0) {
0076 | return 1;
     Meaning: Ends the program with an error status.
0077 | }

```
     Meaning: Ends the current block.
0079 |     for (int start = 0; start <= text_length - sub_length; start++) {
     Meaning: Repeats the block according to its initialization, condition, and
     update.
0080 |         int offset = 0;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0081 |         while (offset < sub_length &&
     Meaning: Repeats the block while the stated condition remains true.
0082 |                 text[start + offset] == substring[offset]) {
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0083 |             offset++;
     Meaning: Increases the loop index or count by one.
0084 |         }
     Meaning: Ends the current block.
0085 |         if (offset == sub_length) {
     Meaning: Runs the following block only when this condition is true: (offse
t
     == sub_length) {
0086 |             return start + 1;
     Meaning: Sends this result back to the function's caller.
0087 |         }
     Meaning: Ends the current block.
0088 |     }
     Meaning: Ends the current block.
0089 |     return 0;
     Meaning: Ends the program successfully.
0090 | }
     Meaning: Ends the current block.
0092 | // Count English vowels and consonants in a string.
     Meaning: Comment: Count English vowels and consonants in a string.
0093 | void my_count_vowels_consonants(const char text[], int *vowels,
     Meaning: Defines my_count_vowels_consonants(): counts English alphabetic
     vowels and consonants.
0094 |                                 int *consonants)
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0095 | {
     Meaning: Begins a block of statements.
0096 |     *vowels = 0;
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0097 |     *consonants = 0;
     Meaning: Algorithm comment: documents the intended steps; the compiler
     ignores it.
0099 |     for (int i = 0; text[i] != '\0'; i++) {
     Meaning: Advances an index one character or element at a time.
0100 |         char letter = text[i];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0101 |         if ((letter >= 'A' && letter <= 'Z') ||
     Meaning: Runs the following block only when this condition is true: ((lett
er
     >= 'A' && letter <= 'Z') ||
0102 |             (letter >= 'a' && letter <= 'z')) {
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0103 |             if (letter == 'a' || letter == 'e' || letter == 'i' ||
     Meaning: Runs the following block only when this condition is true: (lette
r
     == 'a' || letter == 'e' || letter == 'i' ||
```

```
0104 |                 letter == 'o' || letter == 'u' || letter == 'A' ||
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0105 |                 letter == 'E' || letter == 'I' || letter == 'O' ||
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0106 |                 letter == 'U') {
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0107 |                 (*vowels)++;
     Meaning: Increases the loop index or count by one.
0108 |             } else {
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0109 |                 (*consonants)++;
     Meaning: Increases the loop index or count by one.
0110 |             }
     Meaning: Ends the current block.
0111 |         }
     Meaning: Ends the current block.
0112 |     }
     Meaning: Ends the current block.
0113 | }
     Meaning: Ends the current block.
0115 | // Print each group of non-space characters as a separate token.
     Meaning: Comment: Print each group of non-space characters as a separate
     token.
0116 | void my_strtokenize(char text[])
     Meaning: Defines my_strtokenize(): prints each run of non-space characters
     as a token.
0117 | {
     Meaning: Begins a block of statements.
0118 |     int i = 0;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0119 |     int token_number = 1;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0121 |     while (text[i] != '\0') {
     Meaning: Repeats the block while the stated condition remains true.
0122 |         while (text[i] == ' ') {
     Meaning: Repeats the block while the stated condition remains true.
0123 |             i++;
     Meaning: Increases the loop index or count by one.
0124 |         }
     Meaning: Ends the current block.
0125 |         if (text[i] == '\0') {
     Meaning: Runs the following block only when this condition is true: (text[
i]
     == '\0') {
0126 |             break;
     Meaning: Exits the current switch case or loop rather than falling through
.
0127 |         }
     Meaning: Ends the current block.
0129 |         printf("Token %d: ", token_number++);
     Meaning: Prints a prompt or result: “Token %d: ” (format markers are fille
d
     by the arguments).
0130 |         while (text[i] != '\0' && text[i] != ' ') {
     Meaning: Repeats the block while the stated condition remains true.
0131 |             putchar(text[i]);
```

      Meaning: Prints one character from the string.
0132 | i++;
      Meaning: Increases the loop index or count by one.
0133 | }
      Meaning: Ends the current block.
0134 | putchar('\n');
      Meaning: Prints one character from the string.
0135 | }
      Meaning: Ends the current block.
0136 | }
      Meaning: Ends the current block.
0138 | #endif
      Meaning: This line contributes to the surrounding C statement or expressio
n.

END OF FILE NOTES

==
==
p15.c — 23 source lines
==
==

  Purpose: Demonstrates strstr and strncat. strstr returns a pointer into the
  original string at the match; strncat appends up to three characters and alway
s
  writes a final '\0'. The first array has extra capacity for that appended text
.

WALKTHROUGH / EXPECTED RESULT
s1 initially contains "ANIL KUMAR GUPTA"; s2 contains "KUMAR".
strstr finds the match inside s1 and returns a pointer to its K.
Printing that pointer as %s prints from that K to the string end:
"KUMAR GUPTA". strncat then appends the first 3 characters of s2
(KUM) to the end of s1, producing "ANIL KUMAR GUPTAKUM".
The 32-character s1 array is deliberately larger than its initial
text so there is room for the appended characters and '\0'.

LINE-BY-LINE NOTES
0001 | #include <stdio.h>
      Meaning: Imports stdio.h: standard input/output functions such as printf,
      scanf, fgets, and getchar.
0002 | #include <string.h>
      Meaning: Imports string.h: strchr for checking whether an input line
      contains a newline.
0004 | int main(void)
      Meaning: Defines main(): the program entry point; execution starts here.
0005 | {
      Meaning: Begins a block of statements.
0006 | // Leave enough room for the original text, three appended
         characters, and '\0'.
      Meaning: Comment: Leave enough room for the original text, three appended
      characters, and '\0'.
0007 | char s1[32] = "ANIL KUMAR GUPTA";
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0008 | char s2[] = "KUMAR";
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0010 | // strstr returns a pointer to the first occurrence of "KUMAR" in s1.
      Meaning: Comment: strstr returns a pointer to the first occurrence of
      "KUMAR" in s1.

```
0011 |     char *found = strstr(s1, s2);
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0012 |     if (found != NULL) {
      Meaning: Checks whether strstr found the requested substring.
0013 |         printf("%s\n", found);
      Meaning: Prints a prompt or result: “%s\n” (format markers are filled by t
he
      arguments).
0014 |     } else {
      Meaning: This line contributes to the surrounding C statement or expressio
n.
0015 |         printf("Substring not found.\n");
      Meaning: Prints a prompt or result: “Substring not found.\n” (format marke
rs
      are filled by the arguments).
0016 |     }
      Meaning: Ends the current block.
0018 |     // Append the first three characters of s2 to the end of s1.
      Meaning: Comment: Append the first three characters of s2 to the end of s1
.
0019 |     strncat(s1, s2, 3);
      Meaning: Executes this statement; the semicolon marks its end.
0020 |     printf("%s\n", s1);
      Meaning: Prints a prompt or result: “%s\n” (format markers are filled by t
he
      arguments).
0022 |     return 0;
      Meaning: Ends the program successfully.
0023 | }
      Meaning: Ends the current block.

END OF FILE NOTES

================================================================================
==
p16.c — 73 source lines
================================================================================
==

  Purpose: Reads a sentence, separates it into whitespace-delimited substrings,
  counts them, and prints the third one. The sentence in the prompt 'This is a
  sentence.' has four substrings; its third substring is 'a'.

WALKTHROUGH / WHY THE THIRD SUBSTRING IS FOUND
i is the current character position. First skip whitespace. If
the string has not ended, a word begins: increment count. For
words one and two, advance to their ends. For word three, print
characters until the next whitespace. Continue scanning to count
later words too. The cast to unsigned char is the safe way to pass
a possibly signed char to isspace.
Example: "This is a sentence." gives This (1), is (2), a (3),
sentence. (4). So the count is 4 and the third substring is "a".

LINE-BY-LINE NOTES
0001 | #include <ctype.h>
      Meaning: Imports ctype.h: isspace for recognizing whitespace characters.
0002 | #include <stdio.h>
      Meaning: Imports stdio.h: standard input/output functions such as printf,
      scanf, fgets, and getchar.
0004 | #define SENTENCE_SIZE 200
      Meaning: Reserves space for at most 199 input characters plus the string
```

```
     terminator.
0006 | // Count the whitespace-separated substrings and print the third one if
        present.
     Meaning: Comment: Count the whitespace-separated substrings and print the
     third one if present.
0007 | void Num_sstr(const char *sentence)
     Meaning: Defines Num_sstr(): counts whitespace-separated words and prints
     the third word if present.
0008 | {
     Meaning: Begins a block of statements.
0009 |     int count = 0;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0010 |     int i = 0;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0012 |     while (sentence[i] != '\0') {
     Meaning: Repeats the block while the stated condition remains true.
0013 |         // Skip spaces between substrings.
     Meaning: Comment: Skip spaces between substrings.
0014 |         while (sentence[i] != '\0' && isspace((unsigned
        char)sentence[i])) {
     Meaning: Skips whitespace so the next scan begins at a word.
0015 |             i++;
     Meaning: Increases the loop index or count by one.
0016 |         }
     Meaning: Ends the current block.
0017 |         if (sentence[i] == '\0') {
     Meaning: Runs the following block only when this condition is true:
     (sentence[i] == '\0') {
0018 |             break;
     Meaning: Exits the current switch case or loop rather than falling through
     .
0019 |         }
     Meaning: Ends the current block.
0021 |         count++;
     Meaning: Increases the loop index or count by one.
0022 |         if (count == 3) {
     Meaning: Runs the following block only when this condition is true: (count
     == 3) {
0023 |             printf("Third substring: ");
     Meaning: Prints a prompt or result: “Third substring: ” (format markers ar
e
     filled by the arguments).
0024 |             while (sentence[i] != '\0' &&
     Meaning: Repeats the block while the stated condition remains true.
0025 |                    !isspace((unsigned char)sentence[i])) {
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0026 |                 putchar(sentence[i]);
     Meaning: Prints one character from the string.
0027 |                 i++;
     Meaning: Increases the loop index or count by one.
0028 |             }
     Meaning: Ends the current block.
0029 |             putchar('\n');
     Meaning: Prints one character from the string.
0030 |         } else {
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0031 |             // Move to the end of this substring.
     Meaning: Comment: Move to the end of this substring.
```

```
0032 |             while (sentence[i] != '\0' &&
     Meaning: Repeats the block while the stated condition remains true.
0033 |                    !isspace((unsigned char)sentence[i])) {
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0034 |                 i++;
     Meaning: Increases the loop index or count by one.
0035 |             }
     Meaning: Ends the current block.
0036 |         }
     Meaning: Ends the current block.
0037 |     }
     Meaning: Ends the current block.
0039 |     printf("Number of substrings: %d\n", count);
     Meaning: Prints a prompt or result: “Number of substrings: %d\n” (format
     markers are filled by the arguments).
0040 |     if (count < 3) {
     Meaning: Checks whether a third substring exists.
0041 |         printf("There is no third substring.\n");
     Meaning: Prints a prompt or result: “There is no third substring.\n” (form
at
     markers are filled by the arguments).
0042 |     }
     Meaning: Ends the current block.
0043 | }
     Meaning: Ends the current block.
0045 | // Read a sentence and pass its character array to Num_sstr.
     Meaning: Comment: Read a sentence and pass its character array to Num_sstr
.
0046 | void mystr(void)
     Meaning: Defines mystr(): reads the sentence into sntm and passes it to
     Num_sstr.
0047 | {
     Meaning: Begins a block of statements.
0048 |     char sntm[SENTENCE_SIZE];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0049 |     int i = 0;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0051 |     printf("Enter a sentence: ");
     Meaning: Prints a prompt or result: “Enter a sentence: ” (format markers a
re
     filled by the arguments).
0052 |     if (fgets(sntm, sizeof(sntm), stdin) == NULL) {
     Meaning: Checks for end-of-file or an input error while reading a line.
0053 |         printf("Could not read the sentence.\n");
     Meaning: Prints a prompt or result: “Could not read the sentence.\n” (form
at
     markers are filled by the arguments).
0054 |         return;
     Meaning: Sends this result back to the function's caller.
0055 |     }
     Meaning: Ends the current block.
0057 |     // Remove the newline added by fgets, if the sentence fits on the
        line.
     Meaning: Comment: Remove the newline added by fgets, if the sentence fits
on
     the line.
0058 |     while (sntm[i] != '\0' && sntm[i] != '\n') {
     Meaning: Repeats the block while the stated condition remains true.
0059 |         i++;
```

      Meaning: Increases the loop index or count by one.
0060 | }
      Meaning: Ends the current block.
0061 | if (sntm[i] == '\n') {
      Meaning: Runs the following block only when this condition is true: (sntm[
i]
      == '\n') {
0062 | sntm[i] = '\0';
      Meaning: Stores the result of '\0' in sntm[i].
0063 | }
      Meaning: Ends the current block.
0065 | Num_sstr(sntm);
      Meaning: Executes this statement; the semicolon marks its end.
0066 | }
      Meaning: Ends the current block.
0068 | int main(void)
      Meaning: Defines main(): the program entry point; execution starts here.
0069 | {
      Meaning: Begins a block of statements.
0070 | // Start the sentence input and substring count.
      Meaning: Comment: Start the sentence input and substring count.
0071 | mystr();
      Meaning: Executes this statement; the semicolon marks its end.
0072 | return 0;
      Meaning: Ends the program successfully.
0073 | }
      Meaning: Ends the current block.

END OF FILE NOTES

==
==
p17.c — 77 source lines
==
==

  Purpose: Finds matrix rank by Gaussian elimination. It searches for a pivot in
  each column, swaps that row into place, and eliminates entries below it. The
  number of pivots is the rank. EPSILON treats tiny floating point residuals as
  zero.

WALKTHROUGH / WHY PIVOTS GIVE THE RANK
A pivot is a leading nonzero value in a row after elimination.
For each column, choose the largest-magnitude candidate from the
remaining rows; swapping rows does not change rank. If all
candidates are near zero, that column has no pivot. Otherwise
eliminate entries beneath it and count one pivot. At the end,
pivot count = rank, the number of independent rows (also equal
to the number of independent columns).
Example: rows [1 2], [2 4] become [1 2], [0 0]. There is one
nonzero row/pivot, so rank is 1. EPSILON makes this approximate
for floating-point input; very small values may be treated as 0.

LINE-BY-LINE NOTES
0001 | #include <math.h>
      Meaning: Imports math.h: fabs for comparing magnitudes of floating-point
      pivots.
0002 | #include <stdio.h>
      Meaning: Imports stdio.h: standard input/output functions such as printf,
      scanf, fgets, and getchar.
0004 | #define MAX_SIZE 10
      Meaning: Sets the maximum matrix dimension to 10; arrays and input checks

```
     use this limit.
0005 | #define EPSILON 1e-9
     Meaning: Defines a tiny tolerance so floating-point roundoff can be treate
d
     as zero.
0007 | // Use row reduction to count the independent rows (pivots) in the
       matrix.
     Meaning: Comment: Use row reduction to count the independent rows (pivots)
     in the matrix.
0008 | int find_rank(double matrix[MAX_SIZE][MAX_SIZE], int rows, int cols)
     Meaning: Defines find_rank(): uses row elimination and returns the number
of
     pivots.
0009 | {
     Meaning: Begins a block of statements.
0010 |     int rank = 0;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0012 |     // Search each column for a pivot, starting below the previous pivot
       row.
     Meaning: Comment: Search each column for a pivot, starting below the
     previous pivot row.
0013 |     for (int col = 0; col < cols && rank < rows; col++) {
     Meaning: Loops over matrix rows and columns so each relevant cell is
     visited.
0014 |         int pivot_row = rank;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0015 |         for (int row = rank + 1; row < rows; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0016 |             if (fabs(matrix[row][col]) > fabs(matrix[pivot_row][col])) {
     Meaning: Checks whether a floating-point pivot is effectively zero.
0017 |                 pivot_row = row;
     Meaning: Stores the result of row in pivot_row.
0018 |             }
     Meaning: Ends the current block.
0019 |         }
     Meaning: Ends the current block.
0021 |         // Ignore this column if all remaining values are effectively
       zero.
     Meaning: Comment: Ignore this column if all remaining values are effective
ly
     zero.
0022 |         if (fabs(matrix[pivot_row][col]) < EPSILON) {
     Meaning: Checks whether a floating-point pivot is effectively zero.
0023 |             continue;
     Meaning: Skips the rest of this loop pass and starts the next pass.
0024 |         }
     Meaning: Ends the current block.
0026 |         // Move the pivot row into the next rank position.
     Meaning: Comment: Move the pivot row into the next rank position.
0027 |         if (pivot_row != rank) {
     Meaning: Skips the pivot row while eliminating a column.
0028 |             for (int j = 0; j < cols; j++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0029 |                 double temporary = matrix[rank][j];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0030 |                 matrix[rank][j] = matrix[pivot_row][j];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0031 |                 matrix[pivot_row][j] = temporary;
```

```
     Meaning: Stores the result of temporary in matrix[pivot_row][j].
0032 |             }
     Meaning: Ends the current block.
0033 |         }
     Meaning: Ends the current block.
0035 |         // Eliminate the values below this pivot.
     Meaning: Comment: Eliminate the values below this pivot.
0036 |         for (int row = rank + 1; row < rows; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0037 |             double factor = matrix[row][col] / matrix[rank][col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0038 |             for (int j = col; j < cols; j++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0039 |                 matrix[row][j] -= factor * matrix[rank][j];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0040 |             }
     Meaning: Ends the current block.
0041 |         }
     Meaning: Ends the current block.
0043 |         rank++;
     Meaning: Increases the loop index or count by one.
0044 |     }
     Meaning: Ends the current block.
0046 |     return rank;
     Meaning: Sends this result back to the function's caller.
0047 | }
     Meaning: Ends the current block.
0049 | int main(void)
     Meaning: Defines main(): the program entry point; execution starts here.
0050 | {
     Meaning: Begins a block of statements.
0051 |     double matrix[MAX_SIZE][MAX_SIZE];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0052 |     int rows, cols;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0054 |     // Read and validate the matrix dimensions.
     Meaning: Comment: Read and validate the matrix dimensions.
0055 |     printf("Enter the number of rows and columns (1-%d): ", MAX_SIZE);
     Meaning: Prints a prompt or result: “Enter the number of rows and columns
     (1-%d): ” (format markers are filled by the arguments).
0056 |     if (scanf("%d %d", &rows, &cols) != 2 ||
     Meaning: Checks whether scanf successfully read the requested input values
.
0057 |         rows < 1 || rows > MAX_SIZE || cols < 1 || cols > MAX_SIZE) {
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0058 |         printf("Invalid dimensions. Enter row and column counts from 1 to
         %d.\n",
     Meaning: Prints a prompt or result: “Invalid dimensions. Enter row and
     column counts from 1 to %d.\n” (format markers are filled by the arguments
).
0059 |                MAX_SIZE);
     Meaning: Executes this statement; the semicolon marks its end.
0060 |         return 1;
     Meaning: Ends the program with an error status.
0061 |     }
     Meaning: Ends the current block.
0063 |     // Read each matrix value row by row.
```

      Meaning: Comment: Read each matrix value row by row.
0064 | printf("Enter the matrix elements row by row:\n");
      Meaning: Prints a prompt or result: “Enter the matrix elements row by
      row:\n” (format markers are filled by the arguments).
0065 | for (int row = 0; row < rows; row++) {
      Meaning: Moves down matrix rows, stopping at the specified bound.
0066 | for (int col = 0; col < cols; col++) {
      Meaning: Moves across matrix columns, stopping at the specified bound.
0067 | if (scanf("%lf", &matrix[row][col]) != 1) {
      Meaning: Checks whether scanf successfully read the requested input values
      .
0068 | printf("Invalid matrix element.\n");
      Meaning: Prints a prompt or result: “Invalid matrix element.\n” (format
      markers are filled by the arguments).
0069 | return 1;
      Meaning: Ends the program with an error status.
0070 | }
      Meaning: Ends the current block.
0071 | }
      Meaning: Ends the current block.
0072 | }
      Meaning: Ends the current block.
0074 | // Find and display the matrix rank.
      Meaning: Comment: Find and display the matrix rank.
0075 | printf("Rank of the matrix: %d\n", find_rank(matrix, rows, cols));
      Meaning: Prints a prompt or result: “Rank of the matrix: %d\n” (format
      markers are filled by the arguments).
0076 | return 0;
      Meaning: Ends the program successfully.
0077 | }
      Meaning: Ends the current block.

END OF FILE NOTES

==
==
p18.c — 121 source lines
==
==

  Purpose: Solves AX=B using Gauss-Jordan elimination on an augmented matrix
  [A|B]. It checks for inconsistent rows, then distinguishes a unique solution
  from infinitely many solutions. In the latter case it sets free variables to
  zero and prints one example solution.

WALKTHROUGH / HOW AX=B IS CLASSIFIED
If there are E equations and U unknowns, each augmented row stores
U coefficients followed by its B value. Gauss-Jordan elimination
makes each pivot 1 and zeros its column in all other rows.
A row [0 ... 0 | nonzero] says 0 equals a nonzero number, so there
is no solution. If there is no contradiction and every unknown has
a pivot, the solution is unique. If a variable has no pivot, it is
free and there are infinitely many solutions; setting free
variables to zero gives one concrete solution to display.

LINE-BY-LINE NOTES
0001 | #include <math.h>
      Meaning: Imports math.h: fabs for comparing magnitudes of floating-point
      pivots.
0002 | #include <stdio.h>
      Meaning: Imports stdio.h: standard input/output functions such as printf,
      scanf, fgets, and getchar.

```
0004 | #define MAX_SIZE 10
     Meaning: Sets the maximum matrix dimension to 10; arrays and input checks
     use this limit.
0005 | #define EPSILON 1e-9
     Meaning: Defines a tiny tolerance so floating-point roundoff can be treate
d
     as zero.
0007 | // Find and swap a non-zero pivot into the current row.
     Meaning: Comment: Find and swap a non-zero pivot into the current row.
0008 | int swap_pivot_row(double matrix[MAX_SIZE][MAX_SIZE + 1],
     Meaning: Defines swap_pivot_row(): selects the largest candidate pivot,
     swaps rows, and reports if the pivot is usable.
0009 |                    int pivot_row, int column, int row_count)
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0010 | {
     Meaning: Begins a block of statements.
0011 |     int best_row = pivot_row;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0013 |     for (int row = pivot_row + 1; row < row_count; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0014 |         if (fabs(matrix[row][column]) > fabs(matrix[best_row][column])) {
     Meaning: Checks whether a floating-point pivot is effectively zero.
0015 |             best_row = row;
     Meaning: Stores the result of row in best_row.
0016 |         }
     Meaning: Ends the current block.
0017 |     }
     Meaning: Ends the current block.
0019 |     if (fabs(matrix[best_row][column]) < EPSILON) {
     Meaning: Checks whether a floating-point pivot is effectively zero.
0020 |         return 0;
     Meaning: Ends the program successfully.
0021 |     }
     Meaning: Ends the current block.
0023 |     if (best_row != pivot_row) {
     Meaning: Skips the pivot row while eliminating a column.
0024 |         for (int col = 0; col <= MAX_SIZE; col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0025 |             double temporary = matrix[pivot_row][col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0026 |             matrix[pivot_row][col] = matrix[best_row][col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0027 |             matrix[best_row][col] = temporary;
     Meaning: Stores the result of temporary in matrix[best_row][col].
0028 |         }
     Meaning: Ends the current block.
0029 |     }
     Meaning: Ends the current block.
0030 |     return 1;
     Meaning: Ends the program with an error status.
0031 | }
     Meaning: Ends the current block.
0033 | int main(void)
     Meaning: Defines main(): the program entry point; execution starts here.
0034 | {
     Meaning: Begins a block of statements.
0035 |     double matrix[MAX_SIZE][MAX_SIZE + 1] = {0};
     Meaning: Declares a variable or array; the type determines what values it
```

```
      can hold.
0036 |     int pivot_columns[MAX_SIZE];
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0037 |     int equations, unknowns;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0038 |     int rank = 0;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0040 |     // A has one row per equation and one column per unknown; the last
         column stores B.
      Meaning: Comment: A has one row per equation and one column per unknown; t
he
      last column stores B.
0041 |     printf("Enter the number of equations and unknowns (1-%d): ",
         MAX_SIZE);
      Meaning: Prints a prompt or result: “Enter the number of equations and
      unknowns (1-%d): ” (format markers are filled by the arguments).
0042 |     if (scanf("%d %d", &equations, &unknowns) != 2 ||
      Meaning: Checks whether scanf successfully read the requested input values
.
0043 |         equations < 1 || equations > MAX_SIZE ||
      Meaning: This line contributes to the surrounding C statement or expressio
n.
0044 |         unknowns < 1 || unknowns > MAX_SIZE) {
      Meaning: This line contributes to the surrounding C statement or expressio
n.
0045 |         printf("Invalid dimensions. Enter counts from 1 to %d.\n",
         MAX_SIZE);
      Meaning: Prints a prompt or result: “Invalid dimensions. Enter counts from
 1
      to %d.\n” (format markers are filled by the arguments).
0046 |         return 1;
      Meaning: Ends the program with an error status.
0047 |     }
      Meaning: Ends the current block.
0049 |     // Read each equation's coefficients followed by its right-hand-side
         value.
      Meaning: Comment: Read each equation's coefficients followed by its right-
      hand-side value.
0050 |     printf("Enter the coefficients of each equation, followed by its B
         value.\n");
      Meaning: Prints a prompt or result: “Enter the coefficients of each
      equation, followed by its B value.\n” (format markers are filled by the
      arguments).
0051 |     for (int row = 0; row < equations; row++) {
      Meaning: Moves down matrix rows, stopping at the specified bound.
0052 |         printf("Equation %d (%d coefficients and then B): ",
      Meaning: Prints a prompt or result: “Equation %d (%d coefficients and then
      B): ” (format markers are filled by the arguments).
0053 |                row + 1, unknowns);
      Meaning: Executes this statement; the semicolon marks its end.
0054 |         for (int col = 0; col <= unknowns; col++) {
      Meaning: Moves across matrix columns, stopping at the specified bound.
0055 |             if (scanf("%lf", &matrix[row][col]) != 1) {
      Meaning: Checks whether scanf successfully read the requested input values
.
0056 |                 printf("Invalid number in the system.\n");
      Meaning: Prints a prompt or result: “Invalid number in the system.\n”
      (format markers are filled by the arguments).
0057 |                 return 1;
```

```
     Meaning: Ends the program with an error status.
0058 |             }
     Meaning: Ends the current block.
0059 |         }
     Meaning: Ends the current block.
0060 |     }
     Meaning: Ends the current block.
0062 |     // Use Gauss-Jordan elimination to produce pivot columns and reduced
        rows.
     Meaning: Comment: Use Gauss-Jordan elimination to produce pivot columns an
d
     reduced rows.
0063 |     for (int col = 0; col < unknowns && rank < equations; col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0064 |         if (!swap_pivot_row(matrix, rank, col, equations)) {
     Meaning: Runs the following block only when this condition is true:
     (!swap_pivot_row(matrix, rank, col, equations)) {
0065 |             continue;
     Meaning: Skips the rest of this loop pass and starts the next pass.
0066 |         }
     Meaning: Ends the current block.
0068 |         // Make the pivot equal to 1.
     Meaning: Comment: Make the pivot equal to 1.
0069 |         double pivot = matrix[rank][col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0070 |         for (int j = 0; j <= unknowns; j++) {
     Meaning: Repeats the block according to its initialization, condition, and
     update.
0071 |             matrix[rank][j] /= pivot;
     Meaning: Stores the result of pivot in matrix[rank][j] /.
0072 |         }
     Meaning: Ends the current block.
0074 |         // Eliminate this variable from every other equation.
     Meaning: Comment: Eliminate this variable from every other equation.
0075 |         for (int row = 0; row < equations; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0076 |             if (row != rank) {
     Meaning: Skips the pivot row while eliminating a column.
0077 |                 double factor = matrix[row][col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0078 |                 for (int j = 0; j <= unknowns; j++) {
     Meaning: Repeats the block according to its initialization, condition, and
     update.
0079 |                     matrix[row][j] -= factor * matrix[rank][j];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0080 |                 }
     Meaning: Ends the current block.
0081 |             }
     Meaning: Ends the current block.
0082 |         }
     Meaning: Ends the current block.
0084 |         pivot_columns[rank] = col;
     Meaning: Stores the result of col in pivot_columns[rank].
0085 |         rank++;
     Meaning: Increases the loop index or count by one.
0086 |     }
     Meaning: Ends the current block.
0088 |     // A zero coefficient row with a non-zero B value makes the system
        inconsistent.
```

```
     Meaning: Comment: A zero coefficient row with a non-zero B value makes the
     system inconsistent.
0089 |     for (int row = 0; row < equations; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0090 |         int all_zero = 1;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0091 |         for (int col = 0; col < unknowns; col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0092 |             if (fabs(matrix[row][col]) >= EPSILON) {
     Meaning: Checks whether a floating-point pivot is effectively zero.
0093 |                 all_zero = 0;
     Meaning: Stores the result of 0 in all_zero.
0094 |                 break;
     Meaning: Exits the current switch case or loop rather than falling through
     .
0095 |             }
     Meaning: Ends the current block.
0096 |         }
     Meaning: Ends the current block.
0097 |         if (all_zero && fabs(matrix[row][unknowns]) >= EPSILON) {
     Meaning: Checks whether a floating-point pivot is effectively zero.
0098 |             printf("The system has no solution.\n");
     Meaning: Prints a prompt or result: “The system has no solution.\n” (forma
     t
     markers are filled by the arguments).
0099 |             return 0;
     Meaning: Ends the program successfully.
0100 |         }
     Meaning: Ends the current block.
0101 |     }
     Meaning: Ends the current block.
0103 |     // Set free variables to zero to show one solution when infinitely
        many exist.
     Meaning: Comment: Set free variables to zero to show one solution when
     infinitely many exist.
0104 |     double solution[MAX_SIZE] = {0};
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0105 |     for (int row = 0; row < rank; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0106 |         solution[pivot_columns[row]] = matrix[row][unknowns];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0107 |     }
     Meaning: Ends the current block.
0109 |     if (rank == unknowns) {
     Meaning: Checks whether there is a pivot for every unknown.
0110 |         printf("The system has a unique solution:\n");
     Meaning: Prints a prompt or result: “The system has a unique solution:\n”
     (format markers are filled by the arguments).
0111 |     } else {
     Meaning: This line contributes to the surrounding C statement or expressio
     n.
0112 |         printf("The system has infinitely many solutions.\n");
     Meaning: Prints a prompt or result: “The system has infinitely many
     solutions.\n” (format markers are filled by the arguments).
0113 |         printf("One solution, with free variables set to 0, is:\n");
     Meaning: Prints a prompt or result: “One solution, with free variables set
     to 0, is:\n” (format markers are filled by the arguments).
0114 |     }
     Meaning: Ends the current block.
```

```
0116 |     for (int col = 0; col < unknowns; col++) {
      Meaning: Moves across matrix columns, stopping at the specified bound.
0117 |         printf("x%d = %.6g\n", col + 1, solution[col]);
      Meaning: Prints a prompt or result: “x%d = %.6g\n” (format markers are
      filled by the arguments).
0118 |     }
      Meaning: Ends the current block.
0120 |     return 0;
      Meaning: Ends the program successfully.
0121 | }
      Meaning: Ends the current block.

END OF FILE NOTES

================================================================================
p19.c — 85 source lines
================================================================================

  Purpose: Finds a square matrix inverse by reducing [A|I] to [I|A^-1]. Each piv
ot
  is made 1 and the other entries in its column are eliminated. If a pivot canno
t
  be found, the matrix is singular and has no inverse.

WALKTHROUGH / WHY [A|I] BECOMES [I|A^-1]
A inverse exists only for square, nonsingular matrices. Start with
A on the left and identity I on the right. Apply the same legal
row operations to both halves. If the left side can become I,
those operations have transformed the right side into A inverse.
A missing pivot means a row is dependent (determinant zero), so A
is singular and no inverse exists.
Example: A = [[4,7],[2,6]] produces inverse
[[0.6,-0.7],[-0.2,0.4]]. Multiplying A by this result gives I.

LINE-BY-LINE NOTES
0001 | #include <math.h>
      Meaning: Imports math.h: fabs for comparing magnitudes of floating-point
      pivots.
0002 | #include <stdio.h>
      Meaning: Imports stdio.h: standard input/output functions such as printf,
      scanf, fgets, and getchar.
0004 | #define MAX_SIZE 10
      Meaning: Sets the maximum matrix dimension to 10; arrays and input checks
      use this limit.
0005 | #define EPSILON 1e-9
      Meaning: Defines a tiny tolerance so floating-point roundoff can be treate
d
      as zero.
0007 | int main(void)
      Meaning: Defines main(): the program entry point; execution starts here.
0008 | {
      Meaning: Begins a block of statements.
0009 |     double matrix[MAX_SIZE][2 * MAX_SIZE] = {0};
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0010 |     int size;
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0012 |     // Read and validate the matrix order; only square matrices can have
        inverses.
```

```
     Meaning: Comment: Read and validate the matrix order; only square matrices
     can have inverses.
0013 |     printf("Enter the order of the square matrix (1-%d): ", MAX_SIZE);
     Meaning: Prints a prompt or result: “Enter the order of the square matrix
     (1-%d): ” (format markers are filled by the arguments).
0014 |     if (scanf("%d", &size) != 1 || size < 1 || size > MAX_SIZE) {
     Meaning: Checks whether scanf successfully read the requested input values
     .
0015 |         printf("Invalid size. Enter an integer from 1 to %d.\n",
        MAX_SIZE);
     Meaning: Prints a prompt or result: “Invalid size. Enter an integer from 1
     to %d.\n” (format markers are filled by the arguments).
0016 |         return 1;
     Meaning: Ends the program with an error status.
0017 |     }
     Meaning: Ends the current block.
0019 |     // Read the matrix on the left and place the identity matrix on the
        right.
     Meaning: Comment: Read the matrix on the left and place the identity matri
     x
     on the right.
0020 |     printf("Enter the matrix elements row by row:\n");
     Meaning: Prints a prompt or result: “Enter the matrix elements row by
     row:\n” (format markers are filled by the arguments).
0021 |     for (int row = 0; row < size; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0022 |         for (int col = 0; col < size; col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0023 |             if (scanf("%lf", &matrix[row][col]) != 1) {
     Meaning: Checks whether scanf successfully read the requested input values
     .
0024 |                 printf("Invalid matrix element.\n");
     Meaning: Prints a prompt or result: “Invalid matrix element.\n” (format
     markers are filled by the arguments).
0025 |                 return 1;
     Meaning: Ends the program with an error status.
0026 |             }
     Meaning: Ends the current block.
0027 |         }
     Meaning: Ends the current block.
0028 |         matrix[row][size + row] = 1.0;
     Meaning: Stores the result of 1.0 in matrix[row][size + row].
0029 |     }
     Meaning: Ends the current block.
0031 |     // Apply Gauss-Jordan elimination to turn the left half into the
        identity.
     Meaning: Comment: Apply Gauss-Jordan elimination to turn the left half int
     o
     the identity.
0032 |     for (int pivot_col = 0; pivot_col < size; pivot_col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0033 |         int pivot_row = pivot_col;
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0035 |         // Choose the largest available pivot to improve numerical
        stability.
     Meaning: Comment: Choose the largest available pivot to improve numerical
     stability.
0036 |         for (int row = pivot_col + 1; row < size; row++) {
     Meaning: Loops over matrix rows and columns so each relevant cell is
     visited.
0037 |             if (fabs(matrix[row][pivot_col]) >
```

```
     Meaning: Checks whether a floating-point pivot is effectively zero.
0038 |                 fabs(matrix[pivot_row][pivot_col])) {
     Meaning: This line contributes to the surrounding C statement or expressio
n.
0039 |                 pivot_row = row;
     Meaning: Stores the result of row in pivot_row.
0040 |             }
     Meaning: Ends the current block.
0041 |         }
     Meaning: Ends the current block.
0043 |         // A zero pivot means the matrix is singular and has no inverse.
     Meaning: Comment: A zero pivot means the matrix is singular and has no
     inverse.
0044 |         if (fabs(matrix[pivot_row][pivot_col]) < EPSILON) {
     Meaning: Checks whether a floating-point pivot is effectively zero.
0045 |             printf("The matrix is singular, so it has no inverse.\n");
     Meaning: Prints a prompt or result: “The matrix is singular, so it has no
     inverse.\n” (format markers are filled by the arguments).
0046 |             return 0;
     Meaning: Ends the program successfully.
0047 |         }
     Meaning: Ends the current block.
0049 |         // Swap the pivot row into position when needed.
     Meaning: Comment: Swap the pivot row into position when needed.
0050 |         if (pivot_row != pivot_col) {
     Meaning: Skips the pivot row while eliminating a column.
0051 |             for (int col = 0; col < 2 * size; col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0052 |                 double temporary = matrix[pivot_col][col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0053 |                 matrix[pivot_col][col] = matrix[pivot_row][col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0054 |                 matrix[pivot_row][col] = temporary;
     Meaning: Stores the result of temporary in matrix[pivot_row][col].
0055 |             }
     Meaning: Ends the current block.
0056 |         }
     Meaning: Ends the current block.
0058 |         // Divide the pivot row by its pivot so the pivot becomes 1.
     Meaning: Comment: Divide the pivot row by its pivot so the pivot becomes 1
     .
0059 |         double pivot = matrix[pivot_col][pivot_col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0060 |         for (int col = 0; col < 2 * size; col++) {
     Meaning: Moves across matrix columns, stopping at the specified bound.
0061 |             matrix[pivot_col][col] /= pivot;
     Meaning: Stores the result of pivot in matrix[pivot_col][col] /.
0062 |         }
     Meaning: Ends the current block.
0064 |         // Make all other values in this pivot column zero.
     Meaning: Comment: Make all other values in this pivot column zero.
0065 |         for (int row = 0; row < size; row++) {
     Meaning: Moves down matrix rows, stopping at the specified bound.
0066 |             if (row != pivot_col) {
     Meaning: Skips the pivot row while eliminating a column.
0067 |                 double factor = matrix[row][pivot_col];
     Meaning: Declares a variable or array; the type determines what values it
     can hold.
0068 |                 for (int col = 0; col < 2 * size; col++) {
```

      Meaning: Moves across matrix columns, stopping at the specified bound.
0069 | matrix[row][col] -= factor * matrix[pivot_col][col];
      Meaning: Declares a variable or array; the type determines what values it
      can hold.
0070 | }
      Meaning: Ends the current block.
0071 | }
      Meaning: Ends the current block.
0072 | }
      Meaning: Ends the current block.
0073 | }
      Meaning: Ends the current block.
0075 | // The right half now contains the inverse matrix.
      Meaning: Comment: The right half now contains the inverse matrix.
0076 | printf("Inverse matrix:\n");
      Meaning: Prints a prompt or result: “Inverse matrix:\n” (format markers ar
e
      filled by the arguments).
0077 | for (int row = 0; row < size; row++) {
      Meaning: Moves down matrix rows, stopping at the specified bound.
0078 | for (int col = size; col < 2 * size; col++) {
      Meaning: Moves across matrix columns, stopping at the specified bound.
0079 | printf("%10.4f", matrix[row][col]);
      Meaning: Prints a prompt or result: “%10.4f” (format markers are filled by
      the arguments).
0080 | }
      Meaning: Ends the current block.
0081 | printf("\n");
      Meaning: Prints a prompt or result: “\n” (format markers are filled by the
      arguments).
0082 | }
      Meaning: Ends the current block.
0084 | return 0;
      Meaning: Ends the program successfully.
0085 | }
      Meaning: Ends the current block.

END OF FILE NOTES

==
==
FINAL RECAP
==
==
p12: compare answers and count scores.
p13: read, validate, and operate on matrices.
p14 + mystring.h: menu plus hand-written string operations.
p15: find a substring and append a limited number of characters.
p16: count words separated by whitespace and print word three.
p17: rank = number of pivots after elimination.
p18: solve AX=B and classify solution count.
p19: reduce [A|I] to obtain [I|A inverse].

A useful study method: choose one small input, predict the result, then
trace the loops by hand. Compile and run each file separately; every file
has its own main() and should be built as a separate program.